LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 05 – RELASI ANTAR KELAS

NAMA : Ruhfaidah Shirotul Akhiroh

NIM : 103132400019

KELAS : S1SD-05-REG-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

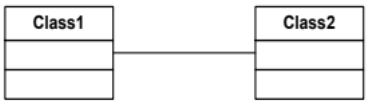
A. Dasar Teori

1. Diagram Kelas

Diagram kelas merupakan sebuah diagram yang digunakan untuk memodelkan kelas-kelas yang digunakan di dalam sistem beserta hubungan antar kelas dalam sistem tersebut. Beberapa elemen penting dalam diagram kelas adalah kelas dan relasi antar kelas. Baris pertama dari simbol diagram kelas menandakan nama dari kelas yang bersangkutan. Baris di bawahnya menyatakan atribut-atribut dari kelas tersebut apa saja, dan baris setelahnya menyatakan method-method yang terdapat pada kelas tersebut.

Untuk menggambarkan hubungan antar kelas digunakan simbol garis antara duakelas yang berelasi. Simbol garis tersebut antara lain:

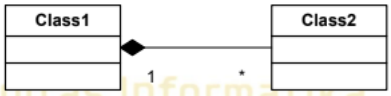
a. Class1 berasosiasi dengan Class2 , digambarkan sebagai berikut:



b. Class2 merupakan elemen part-of dari Class1 (Class1 berelasi agregasi dengan Class2), digambarkan sebagai berikut:



c. Class1 dengan Class2 berelasi komposisi, digambarkan sebagai berikut:



d. Class1 merupakan turunan dari Class2, digambarkan sebagai berikut:



2. Hubungan Antar Kelas

Dalam Object Oriented Programming, kelas-kelas yang terbentuk dapat memiliki hubungan satu dengan yang lainnya, sesuai dengan kondisi dari kelas-kelas yang bersangkutan. Ada beberapa jenis hubungan yang dapat terjadi antara kelas yang satu dengan kelas yang lainnya, antara lain:

a. Asosiasi

Asosiasi merupakan hubungan antara dua kelas terpisah yang terbentuk melalui objek pada tiap kelas. Jika dua kelas dalam suatu model perlu berkomunikasi satu

sama lain, harus ada hubungan di antara kelas hal tersebut dapat diwakili oleh asosiasi.

b. Agregasi

Agregasi merupakan hubungan antara dua kelas yang lebih ketat dan terdapat hubungan bagian (part) dan keseluruhan (whole). Contoh hubungan agregasi ini adalah: Kelas mahasiswa merupakan bagian dari kelas jurusan, tetapi kelas jurusan dan kelas mahasiswa dapat diciptakan sendiri-sendiri.

c. Koposisi

Komposisi merupakan bentuk khusus dari agregasi dimana terdapat hubungan antar kelas yang lebih kuat, karena yang menjadi part (bagian) tidak akan ada tanpa adanya kelas yang menjadi whole.

d. Generalisasi dan Spesialisasi (terkait dengan pewarisan)

B. Guided

1. Asosiasi

```
package com.week5.Guided1;

public class Mahasiswa {
        private String nim;
        private String nama;
        private int nilai;

        public void setNama(String nama) {
            this.nama = nama;
        }

        public void setNim(String nim) {
            this.nim = nim;
        }

        public void setNilai(int nilai){
            this.nilai = nilai;
        }

        public String getNama() {
            return this.nama;
        }

        public String getNim() {
            return this.nim;
        }

        public int getNilai(){
            return this.nilai;
        }
```

```
}
```

```
package com.week5.Guided1;

public class Dosen {
   private String kodeDosen;
   private String namaDosen;

   //Setter
   public void setKodeDosen(String kodeDosen) {
      this.kodeDosen = kodeDosen;
   }

   public void setNamaDosen(String namaDosen) {
      this.namaDosen = namaDosen;
   }

   //Getter
   public String getKodeDosen() {
      return this.kodeDosen;
   }

   public String getNamaDosen() {
      return this.namaDosen;
   }

   public void giveScore(Mahasiswa s, int nilai) {
      // ini asosiasi, method milik class Student dipanggil di class Dosen,
   // tp objek Student tidak menjadi atribut dr class Lecture
   s.setNilai(nilai);
   }

   public int getScore(Mahasiswa s) {
      // ini asosiasi, method milik class Student dipanggil di class Lecture,

   // tp objek Student tidak menjadi atribut dr class Lecture
   return s.getNilai();
   }

   public static void main(String[] args) {
      Mahasiswa m = new Mahasiswa();
      m.setNim("130118383");
      m.setNama("Budi");

      Dosen d = new Dosen();
      d.giveScore(m, 90);

      System.out.println("Nim :" + m.getNim());
      System.out.println("Nama :" + m.getNama());
      System.out.println("Nilai :" + d.getScore(m));
```

```
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ modul-05-r
Nim :130118383
Nama :Budi
Nilai :90
------------------------------------------------
BUILD SUCCESS
------------------------------------------------
Total time:  3.480 s
Finished at: 2026-10-14T10:16:12+07:00
```

2. Agregasi

```
package com.week5.Guided.agregasi;

public class Mahasiswa {
    private String NIM, Nama;

    //Constructor
    public Mahasiswa(String NIM, String Nama){
        this.NIM = NIM;
        this.Nama = Nama;
    }

    //Gentter
    public String getNIM(){
        return NIM;
    }

    public String getNama(){
        return Nama;
    }

    //Setter
    public void setNIM(String NIM){
        this.NIM = NIM;
    }

    public void setNama(String Nama){
        this.Nama = Nama;
    }
}
```

```
package com.week5.Guided.agregasi;

public class Jurusan {
    private static int jumlahMahasiswa;
```

```
    private String kodeJurusan, namaJurusan;
    private Mahasiswa[] daftarMahasiswa = new Mahasiswa[32];

    //Constructor
    public Jurusan(String kodeJurusan, String namaJurusan){
        this.kodeJurusan = kodeJurusan;
        this.namaJurusan = namaJurusan;
    }

    //Behaviour
    public void addMahasiswa(Mahasiswa m){
        this.daftarMahasiswa[jumlahMahasiswa] = m;
        jumlahMahasiswa++;
    }

    public void displayMahasiswa(){
        System.out.println("Kode Jurusan: " + this.kodeJurusan);
        System.out.println("Nama Jurusan: " + this.namaJurusan);

        System.out.println("Daftar Mahasiswa: ");

        for (int i = 0; i < jumlahMahasiswa; i++){
            System.out.println(i+1 + ". " + daftarMahasiswa[i].getNIM() + " " +
daftarMahasiswa[i].getNama());
        }
    }

    //Getter
    public String getKodeJurusan(){
        return kodeJurusan;
    }

    public String getNamaJurusan(){
        return namaJurusan;
    }

    //Setter
    public void setKodeJurusan(String kodeJurusan){
        this.kodeJurusan = kodeJurusan;
    }

    public void setNamaJurusan(String namaJurusan){
        this.namaJurusan = namaJurusan;
    }


}
```

```
package com.week5;
```

```
import com.week5.Guided.agregasi.*;
import com.week5.Guided.komposisi.*;

public class Main {

    public static void main(String[] args) {
        runGuided2();
        //runGuided3();
    }

    public static void runGuided2(){
        Mahasiswa m1 = new Mahasiswa("103132400019", "Ruhfa");
        Mahasiswa m2 = new Mahasiswa("103132400050", "Nur");
        Jurusan prodi = new Jurusan("DS", "Data Science");

        prodi.addMahasiswa(m1);
        prodi.addMahasiswa(m2);

        prodi.displayMahasiswa();
    }
```

Output :

```
Kode Jurusan: DS
Nama Jurusan: Data Science
Daftar Mahasiswa:
1. 103132400019 Ruhfa
2. 103132400050 Nur
------------------------------------------------
BUILD SUCCESS
------------------------------------------------
Total time:  1.283 s
Finished at: 2026-10-14T10:17:55+07:00
------------------------------------------------
```

3. Komposisi

```
package com.week5.Guided.komposisi;

public class Processor {
    private String brand;
    private int kecepatan;

    //constructor
    public Processor(String brand,int kecepatan){
        this.brand = brand;
        this.kecepatan = kecepatan;
    }

    //behaviour
    public void displaySpec(){
        System.out.println(this.brand + ", " + this.kecepatan);
```

```
    }
}
```

```
package com.week5.Guided.komposisi;

public class Monitor {
    private String brand;

    public Monitor(String brand){
        this.brand = brand;
    }

    public void DisplaySpec() {
        System.out.println(this.brand);
    }
}
```

```
package com.week5.Guided.komposisi;

public class Mouse {
    private String brand, tipe;

    public Mouse(String brand, String tipe){
        this.brand = brand;
        this.tipe = tipe;
    }

    public void displaySpec(){
        System.out.println(this.brand + ", " + this.tipe);
    }
}
```

```
package com.week5.Guided.komposisi;

public class Komputer {
    private Processor processor;
    private Monitor monitor;
    private Mouse mouse;

    public Komputer(String brandCpu, int kecepatanCpu, String brandMonitor, String
brandMouse, String tipeMouse){
        this.processor = new Processor(brandCpu, kecepatanCpu);
        this.monitor = new Monitor(brandMonitor);
        this.mouse = new Mouse(brandMouse, tipeMouse);
    }

    public void displaySpecKomputer() {
        System.out.println("=== SPESIFIKASI KOMPUTER ===");
```

```
        this.processor.displaySpec();
        this.monitor.DisplaySpec();
        this.mouse.displaySpec();
    }
}
```

```
package com.week5;

import com.week5.Guided.agregasi.*;
import com.week5.Guided.komposisi.*;

public class Main {

    public static void main(String[] args) {
        //runGuided2();
        runGuided3();
    }

     public static void runGuided3(){
        Komputer pc = new Komputer("Intel Core i5",3200,"LG 24
Inch","Logitech","Wireless");
        pc.displaySpecKomputer();
    }
}
```

Output :

```
=== SPESIFIKASI KOMPUTER ===
Intel Core i5, 3200
LG 24 Inch
- Logitech, Wireless
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  1.782 s
Finished at: 2026-10-14T10:21:23+07:00
------------------------------------------------------------------------
```

C. Unguided

1. Soal Unguided 1

   Sebuah cafe membutuhkan sistem sederhana untuk mencatat dan memproses pesanan pelanggan. Dalam sistem ini, relasi antar kelas dirancang untuk mencerminkan kondisi nyata operasional cafe.

   Pengerjaan soal ini dibagi menjadi 3 bagian tugas utama yang masing-masing melatih satu jenis relasi antar kelas, diikuti dengan pengujian pada kelas utama (`Main`).

Jawaban :

```
package com.week5.Unguided;

public class RincianHarga {
        private double hargaDasar;
        private double pajak;

        public RincianHarga(double hargaDasar){
            this.hargaDasar = hargaDasar;
            this.pajak = (hargaDasar * 10) / 100;  //Pajak 10%
        }

        public double getHargaDasar(){
            return this.hargaDasar;
        }

        public double getPajak(){
            return this.pajak;
        }

        public double getHargaTotal(){
            return this.hargaDasar + this.pajak;
        }
}
```

```
        package com.week5.Unguided;

        public class ItemPesanan {
            private String namaItem;
            private RincianHarga rincian;

            public ItemPesanan(String namaItem, double hargaDasar){
                this.namaItem = namaItem;
                this.rincian = new RincianHarga(hargaDasar);
            }

            public String getNamaItem(){
                return this.namaItem;
            }

            public RincianHarga getRincian(){
                return this.rincian;
            }
}
```

```
        package com.week5.Unguided;

        public class Pesanan {
            private int nomorMeja;
```

```
    private ItemPesanan[] daftarItem;
    private int jumlahItem;

    public Pesanan(int nomorMeja, int kapasitas) {
        this.nomorMeja = nomorMeja;
        this.daftarItem = new ItemPesanan[kapasitas];
        this.jumlahItem = 0;
    }

    // Agregasi: ItemPesanan dibuat di luar, lalu dimasukkan ke sini
    public void tambahItem(ItemPesanan item) {
        if (this.jumlahItem < this.daftarItem.length) {
            this.daftarItem[this.jumlahItem] = item;
            this.jumlahItem++;
        } else {
            System.out.println("Pesanan penuh, item tidak dapat ditambahkan.");
        }
    }

    public void tampilkanPesanan() {
        System.out.println("=== DAFTAR PESANAN CAFE ===");
        System.out.println("Nomor Meja: " + this.nomorMeja);
        for (int i = 0; i < this.jumlahItem; i++) {
            ItemPesanan item = this.daftarItem[i];
            RincianHarga r = item.getRincian();
            System.out.println((i + 1) + ". " + item.getNamaItem()
                    + " - Rp " + r.getHargaTotal()
                    + " (Harga: " + r.getHargaDasar()
                    + ", Pajak: " + r.getPajak() + ")");
        }
    }

    public int getNomorMeja(){
        return this.nomorMeja;
    }

    public ItemPesanan[] getDaftarItem(){
        return this.daftarItem;
    }

    public int getJumlahItem(){
        return this.jumlahItem;
    }
}
```

```
package com.week5.Unguided;

public class Kasir {
    private String namakasir;
```

```
    public Kasir(String namaKasir){
        this.namakasir = namaKasir;
    }

    // Asosiasi: Pesanan hanya diterima sebagai parameter, tidak disimpan sebagai
field
    public void prosesPembayaran(Pesanan pesanan) {
        double totalSubtotal = 0;
        double totalPajak = 0;

        for (int i = 0; i < pesanan.getJumlahItem(); i++) {
            RincianHarga r = pesanan.getDaftarItem()[i].getRincian();
            totalSubtotal += r.getHargaDasar();
            totalPajak += r.getPajak();
        }

        System.out.println("=== STRUK PEMBAYARAN KASIR ===");
        System.out.println("Kasir Bertugas : " + this.namakasir);
        System.out.println("Pesanan        : Meja " + pesanan.getNomorMeja());
        System.out.println("Jumlah Item    : " + pesanan.getJumlahItem());
        System.out.println("Total Subtotal : Rp " + totalSubtotal);
        System.out.println("Total Pajak    : Rp " + totalPajak);
        System.out.println("TOTAL BAYAR    : Rp " + (totalSubtotal + totalPajak));
        System.out.println("==============================");
    }

}
```

```
package com.week5.Unguided;

public class Main {
    public static void main(String[] args) {
        // 1. Buat objek Kasir
        Kasir kasir = new Kasir("Budi");

        // 2. Buat 3 objek ItemPesanan secara independen
        ItemPesanan item1 = new ItemPesanan("Iced Americano", 18000);
        ItemPesanan item2 = new ItemPesanan("Beef Croissant", 25000);
        ItemPesanan item3 = new ItemPesanan("Matcha Latte", 22000);

        // 3. Buat objek Pesanan untuk Meja 4 (kapasitas 5 item)
        Pesanan pesanan = new Pesanan(4, 5);

        // 4. Masukkan item ke dalam pesanan
        pesanan.tambahItem(item1);
        pesanan.tambahItem(item2);
        pesanan.tambahItem(item3);

        // 5. Tampilkan rincian pesanan
        pesanan.tampilkanPesanan();
```

```
            System.out.println();

            // 6. Proses pembayaran
            kasir.prosesPembayaran(pesanan);
        }
}
```

Output :

```
] --- exec:3.6.3:exec (default-cli) @ modul-05-relasi-antar-kelas ---
  === DAFTAR PESANAN CAFE ===
  Nomor Meja: 4
  1. Iced Americano - Rp 19800.0 (Harga: 18000.0, Pajak: 1800.0)
  2. Beef Croissant - Rp 27500.0 (Harga: 25000.0, Pajak: 2500.0)
  3. Matcha Latte - Rp 24200.0 (Harga: 22000.0, Pajak: 2200.0)

  === STRUK PEMBAYARAN KASIR ===
  Kasir Bertugas : Budi
  Pesanan        : Meja 4
  Jumlah Item    : 3
  Total Subtotal : Rp 65000.0
  Total Pajak    : Rp 6500.0
  TOTAL BAYAR    : Rp 71500.0
- ==============================
  ------------------------------------------------------------------------
  BUILD SUCCESS
  ------------------------------------------------------------------------
  Total time:  1.540 s
  Finished at: 2026-10-14T11:44:05+07:00
```

Penjelasan :

a. RincianHarga.java : bagian dalam (komposisi)
   - Pajak tidak dikirim dari luar, tetapi dihitung otomatis: 10% dari harga dasar.
   - Method getHrgaTotal mengembalikan harga setelah pajak. Method getHargaDasar dan getpajak hanya mengembalikan nilai field, karena field-nya private.

b. ItemPesanan.java : komposisi
   - Objek RincianHarga dibuat di dalam constructor ItemPesanan, bukan di main, sehingga membuatnya komposisi karena RincianHraga tidak bisa ada tanpa ItemPesanan. Kalau ItemPesanan dihapus, RincianHarga ikut hilang.
   - Constructor menerima hargaDasar berupa angka biasa, bukan objek RincianHarga. Sehingga dari luar, orang tidak pernah tahu atau menyentuh RincianHarga secara langsung.

c. Pesanan.java : agregasi
   - daftarItem adalah array bertipe ItemPesanan dengan ukuran kapasitas. Pada awalnya semua isinya null, atau kosong.
   - jumlahItem adalah penanda berapa slot array yang sudah terisi. Penanda ini perlu karena daftarItem.length selalu 5 walaupun baru 3 yang terisi.

- Method tambahItem akan mengecek dulu apakah array masih ada ruang, jika masih akan menyimpan item di posisi jumlahItem dan jumlahItem akan bertambah.
- Pada method tampilkanPesanan, loop berjalan sampai jumlahItem, buakan sampai length, agar slot kosong tidak ikut diproses.penomoran akan dimulai dari 1.

d. Kasir.java : asosiasi
   - Objek Pesanan hanya datang lewat parameter saat method dipanggil.
   - Didalam method prosesPembayaran, data pesanan diambil lewat getter, karena fieldnya private, semua harga dasar dan pajak dijumlahkan menggunakan += .
   - Total bayar adalah totalSubtotal + totalPajak.

e. Main.java
   - Membuat kasir dengan nama Budi.
   - RincianHarga otomatis ikut dibuat di dalam tiap item, tanpa ditulis di Main.
   - Membuat pesanan untuk meja 4 dengan kapasitas 5 item.

D. Kesimpulan

Relasi antar kelas adalah cara satu kelas berhubungan dengan kelas lain dan bagaimana hubungan itu digambarkan dalam diagram kelas. Perbedaan relasi terletak pada cara objek terhubung dan seberapa kuat ikatannya.

Pada praktikum, asosiasi, agregasi dan komposisi diterapkan dalam satu program. Komposisi, RincianHarga dibuat di dalam constructor ItemPesanan dan tidak bisa berdiri sendiri. Agregasi, ItemPesanan dibuat diluar, lalu dimasukkan ke array di Pesanan melalui tambahItem, item tetap bisa ada tanpa pesanan. Asosiasi, Pesanan hanya dikirim sebagai parameter ke prosesPembayaran, tidak disimpan sebagai field.